

Req2Test: An Approach for the Automatic Generation of Unit Tests from User Stories Using LLMs

Mattheus da Hora França

Department of Informatics and Statistics, Federal
University of Santa Catarina (UFSC)
Santa Catarina, Florianópolis, Brazil
mattheus.franca@udesc.br

Department of Software Engineering, Santa Catarina State
University (UDESC)
Santa Catarina, Ibirama, Brazil

Fabiane B. Vavassori Benitti

Department of Informatics and Statistics, Federal
University of Santa Catarina (UFSC)
Santa Catarina, Florianópolis, Brazil
fabiane.benitti@ufsc.br

Abstract

Software testing remains an important practice that contributes to software quality, yet it persists as a costly and time-consuming task, particularly in the implementation of test cases, which are often manually derived from functional requirements. In agile development, user stories and their acceptance criteria play a central role in guiding this process. Recent advances in Large Language Models (LLMs) offer promising capabilities for test automation. However, challenges persist when dealing with natural language specifications, ambiguity, and the risk of generating incorrect outputs. To address these challenges, we propose Req2Test, a novel approach for the automated generation of unit tests from user stories without requiring existing source code. The approach leverages LLMs and is guided by a structured test planning process encoded in a domain-specific language (DSL) specifically designed for this purpose. We implemented the Req2Test Tool, which operationalizes the approach and supports its application in real-world scenarios. The tool is tailored to web-oriented software architectures, with a particular focus on domain-driven RESTful API services. We evaluated the approach in three phases. First, a controlled study with 34 participants showed strong functional effectiveness, structural quality, and internal quality, and achieving 91.75% average code coverage with high mutation scores. Second, we used EvoSuite as a baseline for comparison, with Req2Test consistently achieving higher mutation scores. Finally, scalability tests on QuixBugs confirmed near-optimal coverage and autonomous detection of 72.8% of injected faults.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; **Automatic programming**; *Domain specific languages*.

Keywords

automated test generation, unit testing, user stories, LLM, domain-specific language

ACM Reference Format:

Mattheus da Hora França and Fabiane B. Vavassori Benitti. 2026. Req2Test: An Approach for the Automatic Generation of Unit Tests from User Stories Using LLMs. In *The 41st ACM/SIGAPP Symposium on Applied Computing (SAC '26)*, March 23–27, 2026, Thessaloniki, Greece. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3748522.3780013>

1 Introduction

Despite its challenges, software testing remains a key practice for ensuring software quality, though it is time-consuming and costly [3, 23]. Test case creation alone can consume 40–70% of the testing life cycle [22], often performed manually from functional requirements [29, 9]. This manual effort, arising from the difficulty of defining suitable input-output combinations, makes the process error-prone and compromises test quality [30].

Testing contributes not only to product quality [6, 15], but also to analysis, design, and code improvement. Recent advances in Large Language Models (LLMs) have opened new possibilities for supporting software quality. Their code and language capabilities have attracted growing interest in software engineering, enabling tasks such as test case generation, error tracing, and bug localization [16]. Among these tasks, test case generation stands out as particularly relevant in agile contexts, where early and continuous testing is encouraged [6, 20]. In this paradigm, tests are tightly coupled with evolving specifications, which are commonly expressed through user stories [17].

User stories specify roles, goals, business value, and acceptance criteria [17], guiding communication between developers and Product Owners (POs)¹. Acceptance criteria, which define functional and non-functional requirements, are expressed either as rule-based descriptions in natural language or as scenario-based specifications using DSLs such as Gherkin [39, 9]. These criteria provide concrete, testable requirements for deriving test cases and ensuring alignment with user expectations [27, 9].

Although automated techniques can extract conditionals, many test cases still require manual refinement [10]. Existing approaches [9, 34, 31] show limited effectiveness on real-world data, especially with grammatical errors or unseen vocabulary [8, 10].

To address these gaps, this research introduces **a novel approach for the automated generation of unit tests from user stories without requiring existing source code**. The solution



This work is licensed under a Creative Commons Attribution 4.0 International License. SAC '26, Thessaloniki, Greece

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2294-3/2026/03
<https://doi.org/10.1145/3748522.3780013>

¹The Product Owner (PO)[5] translates business needs into software solutions, prioritizes requirements, and validates deliverables.

leverages LLMs and is guided by a structured test planning process encoded in a domain-specific language (DSL) specifically developed for this purpose. The approach has been implemented in a tool, Req2Test, which operationalizes the method and enables its use in practical software engineering contexts, particularly for domain-oriented RESTful APIs.

Its effectiveness was assessed through a rigorous three-phase evaluation, comprising (i) a controlled experiment with 34 participants for human validation of the generated test quality, covering functional effectiveness, structural quality, and perceived internal quality; (ii) a state-of-the-art comparative evaluation against EvoSuite [13], and (iii) a scalability and external validity assessment on benchmark projects [25]. The evaluation employed 16 GQM-based metrics [4] to ensure a systematic, multi-perspective analysis.

This research is guided by the following question: **How can the generation of unit tests from user stories be effectively automated?** The main contributions of this paper are as follows:

- We present a novel approach to automatically generate unit tests from user stories using LLMs without requiring existing source code.
- We design a DSL as a structured template for user stories, serving as standardized input for the approach.
- We implement the proposed approach in a tool named *Req2Test Tool*, enabling its practical application in domain-oriented RESTful APIs.
- We conduct a three-phase evaluation comprising human validation, comparative analysis with a state-of-the-art tool (EvoSuite), and scalability assessment on benchmark projects.
- We measure the success rate of the approach across multiple dimensions using 16 GQM-based metrics, enabling a systematic and multi-perspective analysis.

2 Req2Test Approach

This section presents a novel approach for generating automated unit tests from user stories by leveraging LLMs and a structured test plan. The test plan is constructed using a DSL specifically developed for this research, and its generation relies on the completion of earlier stages, including the creation of test scripts, test data, and test cases. The overall workflow is illustrated in Figure 1 and further detailed in the following sections.

2.1 Notations and Conventions

The proposed approach follows a pipeline structure². To model the process, the Business Process Model and Notation (BPMN)³ is used. **Figure 2** presents the BPMN diagram of the approach, where each task is labeled with a numeric reference, and all data objects are identified. Data objects are grouped as: **Input data (I)**, such as manually provided user stories; **Support data (S)**, representing intermediate elements used during test generation; **Test artifacts (TA)**, denoting non-executable outputs; and **Automated tests (AT)**, which correspond to executable unit test classes generated by the approach.

²A software pipeline [37] is a sequence of stages where data is processed or transformed in order, with each stage consuming the output of the previous one to produce the final result.

³<https://www.omg.org/bpmn/>

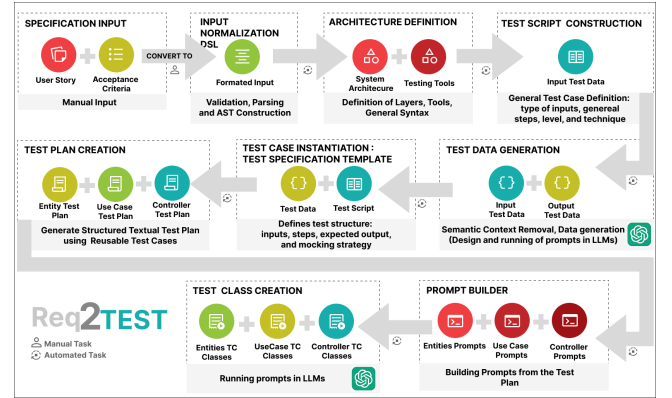


Figure 1: Req2Test Overview.

2.2 Specification

To enable automated test generation, the approach uses a lightweight **Domain-Specific Language (DSL)** to define entities and rule-based acceptance criteria. Written in controlled natural language [14], the DSL ensures both readability and structural precision for validating and transforming input artifacts. The process starts with a list of user stories (I1.1), which are manually adapted into a DSL template, producing two structured inputs: raw entities (I1.2) and raw acceptance rules (I1.3) aligned with the DSL.

The entity template begins with the entity name, followed by a table containing three columns: *Attribute*, *Type*, and *Validation*. Each attribute may include multiple validations, specified in natural language, and can reference other attributes to express contextual constraints. The *Type* field supports both primitive types and references to other entities, allowing relationship modeling. An example is shown in Listing 1. Additionally, cross-attribute validations can be declared after the attribute definitions using the prefix *VALIDATIONS*, formatted as *Name | Natural language rule*. Although the DSL explicitly defines entities, these are derived from the rule-based acceptance criteria originally stated in the user stories. The separation between entities and rules in the DSL was designed to promote reuse across multiple user stories.

Rule-based acceptance criteria follow a predefined structure that starts with the rule identifier and its type: *REF | RuleType*. Additional structured metadata can be declared using the same identifier as a prefix. The supported metadata blocks are:

- *REF.input* — defines the list of input parameters for the rule. If omitted, default inputs inferred from the entity are assumed.
- *REF.output* — defines the expected output values. Default values are assumed if this block is not provided.
- *REF.fields* — lists specific fields used in filtering operations or conditions.
- *REF.validation* — specifies additional validation logic in natural language, complementing the rule description.

This structure promotes modular and reusable specifications that can be composed into higher-level behaviors. A rule definition can be encapsulated and exposed through a controller, which delegates its logic to a use case layer. In this context, an endpoint can be fully defined by instantiating a configured use case.

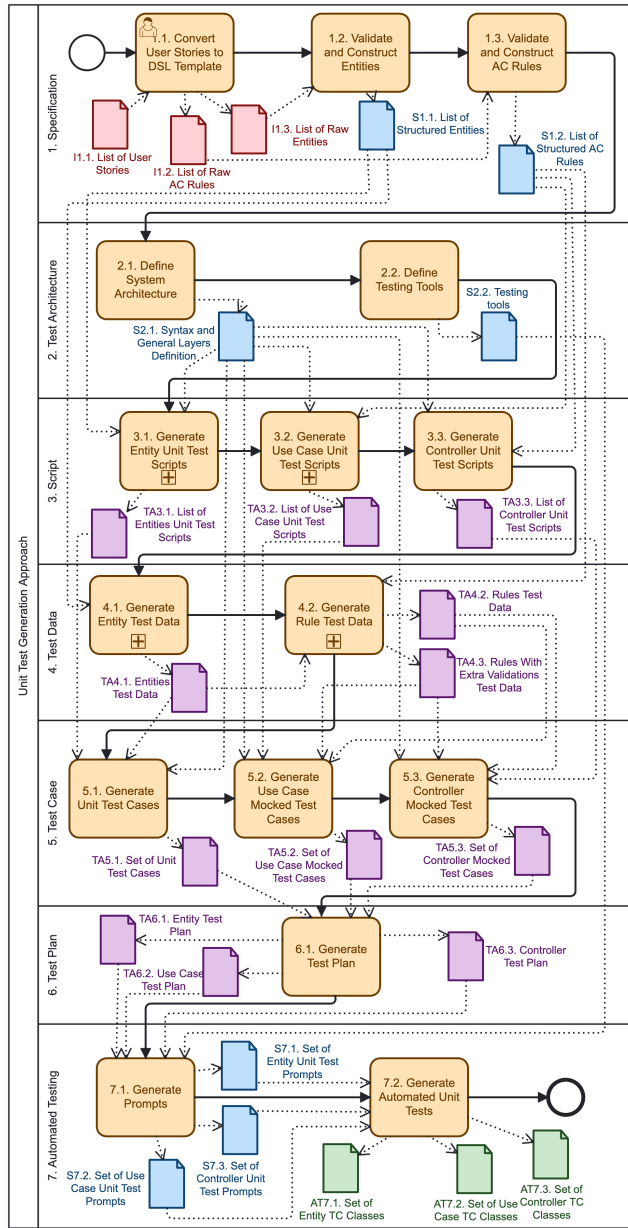


Figure 2: BPMN of the Unit Test Generation Approach.

Listing 1 illustrates three user stories mapped to endpoints: a basic creation, a creation with additional validation, and a filtering operation. The following paragraph describes US1, while US2 and US3 are not detailed below but are defined in Listing 1.

US1: As a user, I want to create a new customer so that they can be registered in the system.

Acceptance Criteria:

- AC01: The system must automatically generate the `idCustomer` as a 36-character UUID.
- AC02: The `fullName` field is required and must contain only letters and spaces.

- AC03: The `birthDate` field is required and the customer must be at least 18 years old.

All three user stories relate to the Customer entity, so their input and output structures are automatically inferred from its definition. When input and output blocks are omitted, they are implicitly derived from the entity's attributes and validations. Implementation details are provided in Section 3.

```
Customer
idCustomer | String | generated: UUID 36 characters [DB]
fullName | String | No special characters; not null
birthDate | Date | Must be at least 18 years old; not null

US1 | endpoint-create | Customer
US2 | endpoint-create | Customer
US2.validation | $birthDate is prior to the year 2000
US3 | endpoint-filter | Customer
US3.fields | fullName, birthDate
```

Listing 1: Example of an entity and rule-based criteria in the DSL format.

2.3 Test Artifact Generation

Following specification, this section presents: the generation of test artifacts including test architecture, test scripts, test data, test cases, test plan, and automated testing.

Test Architecture. This stage defines the system architecture (2.1) and testing tools per layer (2.2), establishing the foundation for test planning and automation. Although testing levels are defined in the scripting stage, this phase outputs syntax conventions, architectural layering (S2.1), and tool selection (S2.2). These artifacts (S5.1) are later combined with test scripts and cases during automation.

Test Scripts. This stage generates test scripts from entities and rules specified earlier. As shown in Figure 2, it includes three main tasks: generating scripts for entities (3.1), use cases (3.2), and controllers (3.3). It uses structured outputs from the specification stage, producing three script types: entity (TA3.1), use case (TA3.2), and controller (TA3.3). These scripts define objectives, targets, steps, levels, and criteria.

Test Data. The test data generation process (4.1) produces structured inputs for unit tests based on entity attributes and validation rules. A carefully designed prompt guides the LLM to produce test data that is relevant, consistent, and valid, minimizing hallucinations. Entities specify all attributes, constraints, and related entities to capture dependencies. Attribute names are abstracted to avoid bias. Input selection uses equivalence partitioning and boundary value analysis [27], with temporal attributes relative to execution time. Outputs include valid and invalid instances labeled with partition class, validity, values, and errors, then normalized and mapped to original names. Next (4.2), test data is generated for business rules (TA4.2), including additional validations (TA4.3), producing three JSON artifacts: entities (TA4.1), use cases (TA4.2), and controllers (TA4.3) supporting textual test case generation.

Test Cases. This stage generates test cases by linking scripts to corresponding test data using predefined templates, reused across entities and projects. As shown in Figure 2, the process involves generating unit test cases (5.1), mocked use case tests (5.2), and mocked controller tests (5.3). Test cases are represented textually, including descriptions, inputs, and expected outputs, linked to scripts

detailing objective, target, steps, level, technique, and input selection. Resulting artifacts are: (TA5.1) unit test cases, (TA5.2) mocked use cases, and (TA5.3) mocked controllers.

Test Plan. This stage produces detailed test plans for entities (TA6.1), use cases (TA6.2), and controllers (TA6.3). Each plan includes all test scripts and their corresponding test cases, outlining the necessary steps for creating automated tests.

Automated Testing. In the final stage, test plans are converted into prompts (7.1) to generate automated unit tests for entities (S7.1), use cases (S7.2), and controllers (S7.3). These prompts are executed to produce the corresponding test artifacts: (AT7.1) for entities, (AT7.2) for use cases, and (AT7.3) for controllers.

3 Implementation: The Req2Test Tool

To demonstrate the feasibility of the proposed approach, we implemented the Req2Test Tool in Kotlin using a modular architecture based on pipes and filters [33]. This style is ideal for processing continuous data streams in sequential stages. In this implementation, we focused on generating unit test cases for a Java-based architecture, covering entities, use cases, and endpoints. The goal was to achieve high unit test coverage for a RESTful API developed with Spring Boot. The test automation relied on widely adopted tools, including JUnit 5, MockMvc, and Mockito. A total of **49 filters** were implemented to support the proposed approach. A full list, with descriptions and source code, is available in the supplementary materials on Github⁴.

Figure 1 summarizes the implementation pipeline. Each filter addresses a specific stage of the test generation workflow. In the Specification stage, input in DSL format are validated and transformed into an Abstract Syntax Tree (AST)[41] by filters like `EntityFilter` and `ACRuleFilter`. The AST guides semantic analysis in later stages. In *Test Script*, test cases are grouped by architectural layer using black- and gray-box strategies. *Test Data Generation* employs LLMs to synthesize consistent data for entities and rules. **Reusable templates (Table 1)** generate unit tests aligned with each entity's structure. Dependencies are mocked to ensure functionality and compliance. Finally, *Test Plan and Automation* consolidates artifacts and uses structured prompts to generate executable test classes with shared data consistency. The reusable test case templates are organized into three levels: Entity, Use Case, and Controller. Entity tests address equivalence partitions and boundary values. Use case tests cover CRUD and filtering operations, including valid and invalid data, duplicates, missing identifiers, and database errors. Controller tests extend these to the API layer, validating HTTP responses (201, 200, 204, 400, 404, 500).

3.1 LLM Integration

We introduced the AIAdapter abstraction to integrate LLMs into the pipeline architecture, enabling prompt execution for individual filters and flexible model selection via dependency injection. Two implementations were developed: `Gpt4Adapter` (OpenAI GPT-4 API) and `OllamaAdapter` (local models such as Llama 3, Mistral, Phi-3 Mini). Although model-agnostic, full pipeline execution used `Gpt4Adapter` for its stability and ease of integration. Other models

were tested in isolation to validate filters and explore cost-effective alternatives but not used end-to-end.

Table 1: Overview of reusable test cases templates

Test Location	Test Cases Descriptions
Entity	
EntityTest	One test per equivalence partition; Lower boundary value; Upper boundary value.
Use Case	
CreateEntityUseCaseTest	Valid and non-duplicate input; Duplicate data handling; DB communication error; Prevent cascade persistence; Invalid data.
UpdateEntityUseCaseTest	Valid update; Invalid data; Nonexistent ID; DB communication error.
GetEntityUseCaseTest	Retrieve valid instance; Instance not found; DB communication error.
GetAllEntityUseCaseTest	Return multiple instances; Return one instance; Return empty; DB communication error.
DeleteEntityUseCaseTest	Valid deletion; Instance not found; DB communication error.
FilterEntityUseCaseTest	Match by all filters; Match by one filter; Null input returns all; No match returns empty; DB communication error.
Controller	
CreateEntityEndpointTest	Valid input (201); Invalid input (400); Internal error (500).
UpdateEntityEndpointTest	Valid update (200); Invalid body (400); Invalid ID (400); Instance not found (404); Internal error (500).
GetEntityEndpointTest	Valid retrieval (200); Instance not found (404); Invalid ID (400); Internal error (500).
GetAllEntityEndpointTest	Return all (200); Internal error (500).
DeleteEntityEndpointTest	Valid deletion (204); Instance not found (404); Invalid ID (400); Internal error (500).
FilterEntityEndpointTest	Valid filter input (200); Internal error (500).
GenericEndpointTest	Valid input (200); Invalid request (400).

3.2 Prompt Engineering

For stages involving LLMs (**Test Data and Automated Testing**), we designed specialized prompts tailored to each automation task, following best practices from recent prompt engineering research [26]. Each prompt was designed with the intention of providing clear operational constraints to the language model, enabling reproducibility, reducing ambiguity, and ensuring alignment with domain-specific expectations. By explicitly encoding rules into the prompt, we intend to minimize model hallucination and improve consistency and usability of the generated outputs across tasks. All prompts used in this study are publicly available in the supplementary materials on **Zenodo**⁵[12]. For dataset generation, the prompt guides the model to produce fully valid and deterministic JSON outputs. It prohibits placeholders, programming expressions, or incomplete constructs, enforcing literal values and precise computation of dependent fields. The test data is generated using software testing techniques such as Equivalence Partitioning and Boundary Value Analysis, with explicit instructions to include null and non-null variations only when permitted. The prompt ensures that all referenced entities remain valid, isolating edge cases exclusively to the main entity under test. An excerpt from the LLM prompt used to generate entity test data is illustrated in Listing 2⁶.

⁴<https://github.com/specgentest/req2test-tool>

⁵Zenodo Supplementary Materials

⁶Only a portion of the dataset generation prompt is shown here for clarity.


```

You are a JSON dataset generator with a strict focus on accuracy, valid syntax,
and
data consistency.
**All output must be valid and strictly conform to JSON syntax**
....
Given an Entity1 with the following attributes:
- var1 as String and with the following validation: [generated: UUID 36
characters ; ]
- var2 as String and with the following validation: [No special characters; not
null; ]
- var3 as Date and with the following validation: [Must be at least 18 years old
;
not null; ]
....
**To apply The Equivalence Partition and Boundary Value Analysis technique,
consider
only the following attributes: {'var2': String'var3': Date} **
....
Create a json with data test only to Entity1. **It must follow the format:

{ 'dataTest': [ { 'className': String, 'isValidInput': Boolean, 'input': {'var2
': String,
'var3': Date}, 'errorMessage': String }}]
....

```

Listing 2: Prompt excerpt used to generate entity test data.

For test case generation, the prompt enforces strict constraints on syntax, structure, and semantics. It mandates the use of specific frameworks and annotations (e.g., @WebMvcTest, @MockBean), adheres to a consistent test structure based on the Arrange-Act-Assert pattern, and ensures naming consistency for methods and exceptions. (All prompts are available on [Zenodo](#)⁷ [12].

4 Evaluation

The evaluation was structured into three complementary phases:

- (i) **Human Validation of Generated Test Quality:** a controlled experiment with 34 participants, assessing functional effectiveness, structural quality, and perceived internal quality.
- (ii) **Comparative Technical Evaluation:** a state-of-the-art comparison against EvoSuite [13], focusing on structural coverage and mutation analysis.
- (iii) **Scalability and External Validity Assessment:** an evaluation on benchmark projects to assess generalization and performance at scale.

Research Questions and Metrics. To systematically explore the effectiveness of our approach, we structured the investigation around specific research questions. **Table 2** presents these questions along with their corresponding metrics. Following the GQM paradigm [4], we defined five goals, which guided the formulation of 15 research questions and 17 associated metrics. In the table 2, qualitative metrics (Likert-based) are highlighted in **blue**, while quantitative metrics are highlighted in **green**. Artifacts and detailed descriptions of the three evaluation phases are publicly available in the supplementary materials on [Zenodo](#)⁸ [12].

4.1 Phase I: Human Validation (Goals 1, 2, 3)

Objective and Method. For the controlled experiment with human participants, we adopted the methodology of Wohlin et al. [38], which provides a comprehensive framework for planning, execution, analysis, and interpretation. In this experiment, we applied

Table 2: Evaluation Questions and Metrics by Quality Goal

ID	Question	Associated Metrics
Goal 1: Assess the functional quality of the automated tests		
Q1.1	Do the tests cover the input boundaries of entities?	(M1) Attribute boundary tests
Q1.2	Do the tests cover the main functionality flows?	(M2) Main flow coverage, (M3) Error detection rate (%)
Q1.3	Do the tests cover the alternative flows?	(M4) Alternative flow coverage, (M3) Error detection rate (%)
Q1.4	Do the tests cover error flows and exception handling?	(M5) Error flow coverage, (M3) Error detection rate (%)
Q1.5	Are the tests effective in detecting functional failures?	(M6) Test confidence, (M7) Mutation score (%), (M8) Code coverage (%)
Goal 2: Assess the structural quality of the automated tests		
Q2.1	Do the tests conform to the system's architecture?	(M9) Test conformance to architecture
Q2.2	Are the tests well-isolated and independent?	(M9) Test conformance to architecture, (M10) Test isolation
Q2.3	Are the tests effective in detecting structural failures?	(M7) Mutation score (%), (M8) Code coverage (%)
Goal 3: Assess the perceived internal quality of the automated tests		
Q3.1	Are test names clear and do they indicate the purpose?	(M11) Clarity of test names
Q3.2	Are the present asserts sufficient and meaningful?	(M12) Sufficiency of asserts
Q3.3	Do error messages help identify the root cause?	(M13) Quality of error messages
Q3.4	Are the tests easy to maintain and modify?	(M14) Ease of maintenance
Q3.5	Are the generated tests relevant and well-targeted?	(M15) Relevance of generated tests
Goal 4: Assess the technical robustness via state-of-the-art comparison		
Q4.1	Is the technical rigor of the DSL-generated tests comparable to that of code-driven tools (EvoSuite)?	(M7) Mutation score (%), (M8) Code coverage (%) (Collected for both Req2Test Tool and EvoSuite)
Goal 5: Assess external validity		
Q5.1	Does the tool demonstrate effective scalability in generating test artifacts for benchmarks?	(M16) Pass rate (%)

Blue: Qualitative metrics (Likert), Green: Quantitative metrics

GQM Goals 1, 2, and 3 from Table 2 to assess functional effectiveness, structural quality, and perceived internal quality. We recruited 34 participants (developers and software engineering practitioners) to inspect and evaluate the test artifacts generated by the Req2Test tool across four projects (P_1 to P_4) - User stories and detailed descriptions are available on [Zenodo](#)⁹ [12], and the implementation is available on Github¹⁰. The projects were deliberately constrained in size (Table 3) to allow for complete human inspection of every generated test case, ensuring high internal validity. Both qualitative and quantitative metrics were employed to provide a comprehensive, multi-perspective assessment of the generated tests. **Experiment Operation:** The experiment was conducted in three sessions with 34 participants (8, 15, and 11 per session). **Table 3** summarizes the four projects considered in the experiment. It lists the lines of code (LOC), number of classes, user stories (US) addressed in each project, and the total number of test cases generated using the Req2Test Tool. The generated test cases are organized by scope: **entity** (data structure and validation rules), **use case** (UC, business logic and process rules), and **endpoint** (EP, controller-level

⁷Prompts - Zenodo Supplementary Materials

⁸Evaluation - Zenodo Supplementary Materials

⁹User stories - Zenodo Supplementary Materials

¹⁰<https://github.com/specgentest/dataset-api>

tests validating API behavior). The complete controlled experiment process is illustrated in **Figure 3**.

Pre-experiment Phase. Projects were instrumented with JaCoCo and PIT to collect code coverage and mutation score metrics, and then duplicated with manually injected bugs to create versions both with and without the generated tests. **Execution Phase.** Participants first completed a background form and then analyzed the buggy projects manually. They executed the Req2Test-generated unit tests, corrected detected faults, and reran the tests to confirm fixes and assess effectiveness. Tests were also executed on clean versions to detect false positives, and participants reviewed test coverage and evaluated the clarity and relevance of the generated tests. Finally, a Likert-scale survey was conducted to collect subjective feedback on test quality and perceived usefulness.

Table 3: Summary of Projects

Proj ID	LOC	Classes	User Stories (US)	Generated Unit Tests
P_1	120	8	Customer: create (US1), get (US2), delete (US3), filter (US4)	38 (Entity: 11 , UC: 14 , EP: 13)
P_2	155	12	Employee: create (US1); Support-Request: create (US2), update (US3), filter (US4)	47 (Entity: 18 , UC: 16 , EP: 13)
P_3	95	6	Customer: create (US1); Account: create (US2); filter (US3), withdraw (US4)	38 (Entity: 16 , UC: 12 , EP: 10)
P_4	140	10	Customer: create (US1); Reservation: create (US2), create with validations (US3), close (US4)	41 (Entity: 23 , UC: 8 , EP: 10)

Statistical Analysis. Likert-based metrics. For the 12 metrics in Table 2, we tested whether the median score exceeds 3.5, indicating positive perception, using a Wilcoxon signed-rank test for non-parametric ordinal data. **Mutation score.** We adopted 70% as the threshold for a medium mutation score [19]. To assess effectiveness, we tested whether Req2Test-generated tests exceed this threshold using a Wilcoxon signed-rank test. Variation across projects was analyzed via one-way ANOVA to detect differences in mean scores. **Code coverage.** For structural metrics (lines, branches, and conditions), we tested whether coverage exceeds 70% [18] using a one-tailed one-sample t -test. Variation across projects was analyzed via one-way ANOVA to detect differences in mean coverage. **Error detection rate.** We tested whether the median error rate is above zero, indicating that the automated approach finds more bugs than the manual one, using a Wilcoxon signed-rank test.

Results. In Functional Quality (Goal 1) all six Likert-based metrics (M1–M6) (Figure 4a) showed statistically significant positive agreement ($p < 0.05$). The **error detection rate (M3)** (Table 4e) confirmed that automated tests revealed more bugs than manual inspection. **Code coverage (M8)** averaged 85.97%, significantly above the 70% threshold, with project-level variation due to complexity of endpoints. **Mutation score (M7)** averaged 81.91%, evidencing strong fault detection, with no significant variation across projects. **Structural Quality (Goal 2).** Architectural conformance (M9) and test isolation (M10) had strong agreement (both $p < 10^{-6}$ - Figure 4b). Structural adequacy was supported by mutation and coverage results (M7–M8) Figures 4d and 4f, reinforcing alignment with architecture and independence. **Perceived Internal Quality**

(Goal 3). Participants strongly agreed on test clarity, assertion sufficiency, error messages, maintainability, and relevance (M11–M15, all $p < 0.05$ - Figure 4c). Clarity of test names (M11) had the highest scores, while assertion sufficiency (M12) and error messages (M13) were slightly lower but still positive.

4.2 Phase II: Req2Test vs. EvoSuite (Goal 4)

Objective and Method. Phase II aims to assess whether test generation from DSL specifications using Req2Test maintains technical quality relative to the state-of-the-art tool **EvoSuite**, which serves as the **baseline**. We compared **Req2Test** with **EvoSuite** across four projects (P_1 – P_4) (table 3) at the package level, focusing on Entity/Model, UseCase, Exception, and Controller. Five independent runs per tool were executed, collecting coverage (%) and mutation score (%) for each package. In this process, **805** tests were generated by Req2Test and **1,051** by EvoSuite. As EvoSuite could not generate tests for Controller due to framework limitations, only Req2Test is reported for this package. Details are provided in **Tables 4 and 5**.

Statistical Analysis. We benchmarked Req2Test using the **Two One-Sided Tests (TOST)** procedure for **coverage (M8)** and **mutation score (M7)** with an equivalence margin of $\Delta = 5.0$ p.p., classifying results as “Equivalent” or “Not equivalent”. Means and standard deviations were computed per project and package, and effect magnitudes were estimated with **Hedges’ g** , corrected for small-sample bias. The analysis used a paired design ($N = 5$ per project, $N = 20$ overall) and interpreted Hedges’ g using standard magnitude thresholds. Values were omitted when the paired standard deviation was zero. **Note on Extreme Hedges’ g Values:** exceptionally high $|g|$ values (> 10) arise from near-zero variance ($\sigma \approx 0$), inflating standardized effects; such cases were retained for completeness but interpreted with caution.

Results. Req2Test achieved coverage (M8) and mutation score (M7) comparable to or higher than EvoSuite. The **Exception** package achieved 100% coverage and mutation in both tools and is omitted from the table due to its small size. For coverage (**M8**) (Table 4), TOST indicated **equivalence** in the **Model** package overall ($N = 20$), with EvoSuite slightly superior (100.0 vs. 93.6). Projects P_1 – P_2 were equivalent, while P_3 – P_4 were not, favoring EvoSuite. The overall effect size ($g = -1.84$) was **very large**, with some extreme values ($g = -13.92$ in P_4 Model) due to near-zero variance inflating g . In contrast, the **UseCase** package showed consistent **non-equivalence** across all projects, with Req2Test outperforming EvoSuite (80.5 vs. 50.4; $g = 2.23$, **extremely large effect**). For the **Controller** package, EvoSuite failed to generate tests, making **Req2Test the only functional tool** for this package.

In **mutation (M7)** analysis (Table 5), TOST identified the **Model** package as **equivalent** overall ($N = 20$, 91.2 vs. 95.7), with individual projects showing mixed results— P_1 and P_4 favoring Req2Test, P_2 and P_3 favoring EvoSuite. The overall effect size ($g = -0.36$) indicates a **small**, practically negligible difference. In contrast, the **UseCase** package was consistently **non-equivalent**, with Req2Test achieving much higher mutation scores (71.8 vs. 22.9; $g = 3.48$, **extremely large effect**). For the **Controller** package, EvoSuite again failed to produce tests, confirming **Req2Test’s clear superiority** in this module.

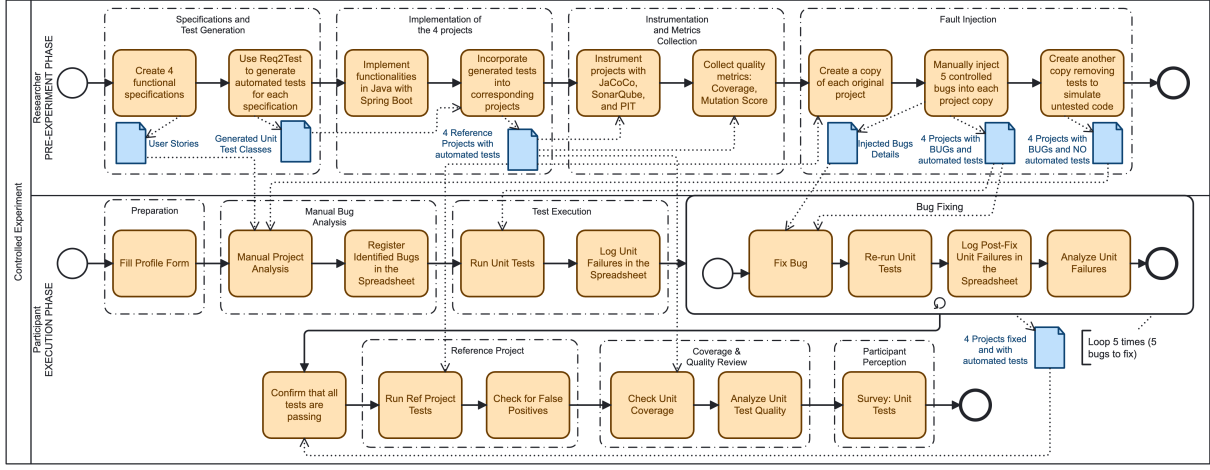


Figure 3: Complete Controlled Experiment Process.

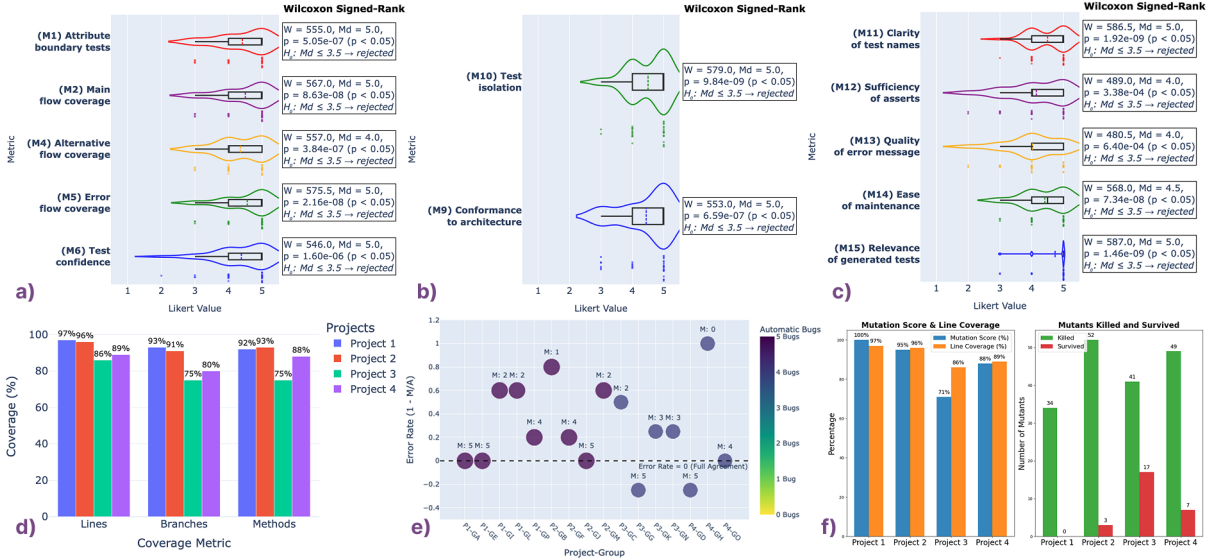


Figure 4: a) Perception metrics for Goal 1. b) Perception metrics for Goal 2. c) Perception metrics for Goal 3. d) Coverage metrics for Goals 1 and 2. e) Error detection rate for Goal 1. f) Mutation scores for Goals 1 and 2.

4.3 Phase III: Scalability and External Validity Assessment (Goal 5)

Objective and Method. Experiments were conducted on 24 algorithms from the *QuixBugs* benchmark [25], comparing the tests automatically generated by *Req2Test* with the manually written tests originally provided in the repository. Each scenario (correct and buggy versions) was executed **five times**, totaling **823 and 790 test cases**, respectively. Details are shown in Tables 6 and 7.

Statistical Analysis. The statistical analysis for both experimental scenarios employed a one-sample *t*-test framework, comparing *Req2Test*'s performance (μ_{R2T}) over $N = 5$ runs against the fixed *QuixBugs* baseline (μ_{QB}) (24 algorithms). Practical equivalence was

evaluated using the **Two One-Sided Tests (TOST)** procedure focusing on **pass rate (M16)** with an equivalence margin of $\Delta = 5.0$ p.p. We also report the 95% confidence interval (CI) for μ_{R2T} , providing additional insight into the stability and precision of the observed mean performance.

Results. For the **correct algorithms** (Table 6), *Req2Test* achieved an average **pass rate (M17)** of **96.0%** (standard deviation = 1.6% across $N = 5$ runs), compared to the highly optimized *QuixBugs* baseline of **99.4%**. The observed difference is -3.4 percentage points (p.p.) in favor of *QuixBugs*. The 95% confidence interval for *Req2Test* ([94.0%, 98.0%]) confirms a statistically significant difference. TOST for practical equivalence were performed using a predefined margin of $\Delta = \pm 5.0$ p.p. This test yielded a *p*-value of **0.0457**. Since $p < 0.05$, we formally declare practical equivalence between

Table 4: Req2Test vs. EvoSuite Coverage (M8)

Proj.	Package	Req2Test ($\mu \pm \sigma$)	EvoSuite ($\mu \pm \sigma$)	Hedges' g	TOST
P1	Model	97.0 $\pm$ 0.0	100.0 $\pm$ 0.0	N/A	Eq.
	UseCase	95.0 $\pm$ 0.0	73.0 $\pm$ 7.9	2.23	Not
	Controller	94.0 $\pm$ 0.0	N/A	N/A	N/A
P2	Model	96.2 $\pm$ 1.1	100.0 $\pm$ 0.0	-2.77	Eq.
	UseCase	96.0 $\pm$ 0.0	51.0 $\pm$ 0.0	N/A	Not
	Controller	91.0 $\pm$ 0.0	N/A	N/A	N/A
P3	Model	89.0 $\pm$ 0.0	100.0 $\pm$ 0.0	N/A	Not
	UseCase	63.0 $\pm$ 0.0	48.0 $\pm$ 0.0	N/A	Not
	Controller	92.0 $\pm$ 0.0	N/A	N/A	N/A
P4	Model	92.2 $\pm$ 0.4	100.0 $\pm$ 0.0	-13.92	Not
	UseCase	68.0 $\pm$ 0.0	29.4 $\pm$ 1.3	22.96	Not
	Controller	79.0 $\pm$ 0.0	N/A	N/A	N/A
Overall	Model	93.6 $\pm$ 3.3	100.0 $\pm$ 0.0	-1.84	Eq.
	UseCase	80.5 $\pm$ 15.5	50.4 $\pm$ 16.3	2.23	Not
	Controller	89.0 $\pm$ 6.8	N/A	N/A	N/A

Hedges' g: positive values favor Req2Test; magnitude **thresholds**—negligible (< 0.2), small ($0.2-0.5$), medium ($0.5-0.8$), large ($0.8-1.2$), very large ($1.2-2.0$), extremely large (≥ 2.0).
TOST: $\delta = \pm 5\%$ evaluates practical equivalence; N/A indicates zero variance or missing data;
Eq. stands for “Equivalent” and **Not** stands for “Not equivalent”.

Table 5: Req2Test vs. EvoSuite Mutation Score (M7)

Proj.	Package	Req2Test ($\mu \pm \sigma$)	EvoSuite ($\mu \pm \sigma$)	Hedges' g	TOST
P1	Model	100.0 $\pm$ 0.0	93.8 $\pm$ 6.3	0.78	Not
	UseCase	100.0 $\pm$ 0.0	45.8 $\pm$ 6.3	6.91	Not
	Controller	94.0 $\pm$ 0.0	N/A	N/A	N/A
P2	Model	87.6 $\pm$ 1.3	95.4 $\pm$ 5.1	-1.37	Not
	UseCase	80.0 $\pm$ 0.0	20.0 $\pm$ 0.0	N/A	Not
	Controller	91.0 $\pm$ 0.0	N/A	N/A	N/A
P3	Model	77.0 $\pm$ 4.2	100.0 $\pm$ 0.0	-4.33	Not
	UseCase	45.0 $\pm$ 0.0	18.0 $\pm$ 0.0	N/A	Not
	Controller	92.0 $\pm$ 0.0	N/A	N/A	N/A
P4	Model	100.0 $\pm$ 0.0	93.2 $\pm$ 2.9	1.70	Not
	UseCase	62.0 $\pm$ 0.0	8.0 $\pm$ 0.0	N/A	Not
	Controller	79.0 $\pm$ 0.0	N/A	N/A	N/A
Overall	Model	91.2 $\pm$ 10.1	95.7 $\pm$ 4.9	-0.36	Eq.
	UseCase	71.8 $\pm$ 21.0	22.9 $\pm$ 14.6	3.48	Not
	Controller	89.0 $\pm$ 6.8	N/A	N/A	N/A

Hedges' g: positive values favor Req2Test; magnitude **thresholds**—negligible (< 0.2), small ($0.2-0.5$), medium ($0.5-0.8$), large ($0.8-1.2$), very large ($1.2-2.0$), extremely large (≥ 2.0).
TOST: $\delta = \pm 5\%$ evaluates practical equivalence; N/A indicates zero variance or missing data;
Eq. stands for “Equivalent” and **Not** stands for “Not equivalent”.

Table 6: Req2Test vs. QuixBugs on 24 Correct Algorithms

Execution	Total Tests	Passed	Failed	Pass Rate (%)
QuixBugs (Baseline)	172	171	1	99.4
Run 1	168	164	4	97.6
Run 2	164	156	8	95.1
Run 3	165	156	9	94.5
Run 4	165	156	9	94.5
Run 5	161	158	3	98.1
Total (Run 1-5)	823	790	33	
Req2Test ($\mu \pm \sigma$)	<i>Based on Pass Rate ($N = 5$ Runs):</i>			96.0 $\pm$ 1.6
95% CI	<i>for Req2Test Pass Rate:</i>			[94.0, 98.0]
Observed Diff. ($\mu_{R2T} - \mu_{QB}$)				3.4 p.p.
Equivalence Margin (Δ)				5.0 p.p.
TOST (p-value)				0.0457

Table 7: Req2Test vs. QuixBugs on 24 Buggy Algorithms

Execution	Total Tests	Passed	Failed	Pass Rate (%)
QuixBugs (Baseline)	171	43	128	25.1
Run 1	164	72	92	43.9
Run 2	156	77	79	49.4
Run 3	156	66	90	42.3
Run 4	156	73	83	46.8
Run 5	158	71	87	44.9
Total (Run 1-5)	790	359	431	45.5
Req2Test ($\mu \pm \sigma$)	<i>Based on Pass Rate ($N = 5$ Runs):</i>			45.5 $\pm$ 2.7
95% CI for Req2Test Pass Rate:				[42.1, 48.9]
Observed Diff. ($\mu_{R2T} - \mu_{QB}$)				+20.4 p.p.
Equivalence Margin (Δ)				5.0 p.p.
TOST (p-value)				> 0.50

Req2Test and the QuixBugs baseline. For the **buggy algorithms** (Table 7), Req2Test obtained a mean **Pass Rate (M17)** of **45.5%** ($\mu \pm \sigma : 45.5 \pm 2.7$), while the QuixBugs baseline reached **25.1%**. Given that a failed test indicates successful fault detection, this means Req2Test identified **54.5%** of the defects ($100\% - 45.5\%$), compared to the **74.9%** detection rate achieved by QuixBugs. The large **+20.4** percentage point difference in Pass Rate in favor of QuixBugs is expected, as its tests were manually designed with prior knowledge of known faults, whereas Req2Test generates test cases **without any information about the code or injected bugs**. Despite this significant disadvantage, Req2Test still achieved approximately **72.8%** ($54.5/74.9$) of the manual benchmark’s detection rate, highlighting its ability to reveal faults.

5 Discussion

The evaluation results provide strong evidence of the technical robustness and practical applicability of the proposed DSL-based approach for automated test generation. In the human validation phase, the generated test suites demonstrated strong technical quality and practical relevance. High scores in error-flow coverage, mutation, and code coverage confirmed that Req2Test effectively exercises application logic and detects faults. Participants rated test isolation and architectural conformance highly, indicating well-structured and modular tests aligned with system layers. Clarity, maintainability, and relevance were also positively evaluated, showing that the tests are understandable and easy to evolve.

In the comparative phase, EvoSuite was used as the baseline for assessing technical rigor. While EvoSuite achieved slightly higher code coverage in the *Model* package, Req2Test obtained consistently higher mutation scores, suggesting that DSL-driven assertions capture semantic faults more effectively. The *UseCase* package reinforced this advantage, with Req2Test surpassing EvoSuite in both coverage and mutation. The lower coverage observed in Projects 3 and 4 was expected, as their specific DSL definitions inherently generate fewer test cases. Overall, Req2Test delivered performance equivalent or superior to the baseline, while also extending automation to layers unsupported by EvoSuite, confirming the robustness of DSL-based test generation.

Finally, in the scalability assessment using the QuixBugs benchmark, Req2Test achieved near-optimal coverage on correct algorithms and solid fault-detection capability on buggy ones, without

manual intervention. The small difference of 3.4 p.p. from the hand-crafted QuixBugs suite was statistically negligible, indicating that Req2Test produces high-quality and reproducible test suites with performance practically equivalent to manually designed tests (coefficient of variation = 1.7%). Despite lacking prior knowledge of injected faults, Req2Test autonomously detected about 72.8% of them, demonstrating scalability and potential for industrial-scale systems. Together, these findings validate Req2Test as a technically rigorous and scalable solution that advances the automation of test generation beyond traditional code-driven approaches.

6 Threats to Validity

The evaluation comprised three phases: human validation (Phase 1), comparative assessment against EvoSuite (Phase 2), and scalability testing on the QuixBugs benchmark (Phase 3). **Construct Validity.** *Phase 1.* Generated tests were manually reviewed and five controlled faults (logic, boundary, initialization, control flow, and data handling) were injected to simulate realistic errors. *Phase 2.* Both tools ran under comparable conditions: EvoSuite, used as baseline, executed with default parameters, while Req2Test results may vary slightly due to the stochastic nature of the underlying LLM. Minor adaptations were needed to run EvoSuite on Spring Boot projects without altering logic or test semantics. **Internal Validity.** *Phase 1.* Thirty-four participants were randomly assigned to 16 groups. Unit tests were gradually released to prevent bias in fault detection. The experiment followed Wohlin et al.'s guidelines [38] and participants interacted with the codebase and generated tests for about three hours, ensuring reliable qualitative assessments. **External Validity.** *Phase 1.* Four experimental RESTful APIs of similar size and structure were used, representative of typical enterprise systems. *Phase 3.* Scalability and generalization were further assessed using 24 of 40 QuixBugs algorithms compatible with the designed DSL. This combination supports realism, diversity, and broader applicability. **Conclusion Validity.** *Phase 1.* Non-parametric Wilcoxon tests were applied. *Phases 2 and 3.* The TOST procedure assessed equivalence, with effect sizes and coefficients of variation quantifying stability. Despite stochastic LLM behavior, consistent results across runs reinforce the reliability and robustness of the conclusions. **Data and Materials Availability.** Detailed descriptions of all evaluation phases are available on Zenodo [12], while the four experimental projects and the Req2Test implementation are publicly accessible on GitHub.¹¹

7 Related Work

Test case generation is central to software testing, with many works exploring automated techniques based on structural and behavioral information. Tools like EvoSuite [13] and Randoop [28] have been evaluated for fault detection on real bugs in academic [32] and industrial [2] contexts. While prior techniques depend on an existing system under test (SUT), this work aims to generate tests prior to system implementation.

Test Generation using LLMs. Recent studies have increasingly explored how LLMs can automate testing across various levels and contexts. They have demonstrated potential in tasks such as test

generation, error tracing, and bug localization [24]. Despite their promise in automating unit test generation, challenges regarding test correctness and effectiveness remain [40]. Moreover, applications extend beyond unit testing, encompassing REST API [21] and end-to-end test generation [1]. Complementing these recent studies, a survey [36] provides a comprehensive review of the use of LLMs in software testing. It analyzes 102 relevant studies from both software testing and LLM perspectives, grouping them into categories. Among these, 20 focus on unit test case generation and 23 on system test input generation, which are the most relevant to our context. While existing studies widely apply LLMs for test generation, this work leverages them in targeted stages, using a DSL to guide data generation and translate test cases into executable tests.

Requirement-based Test Generation. Several approaches leverage requirements artifacts for automated test generation to reduce manual effort and improve coverage. In particular, AGORA uses LLMs to derive acceptance tests from use cases [7], while UMTG creates executable system-level tests from use case specifications and domain models without restricting expressiveness [35]. Tools like Specmate generate tests from user stories' acceptance criteria and lightweight models [9]. The CiRA initiative provides techniques to generate minimal test sets from informal requirement conditionals [11]. While prior work uses specifications as a basis for test generation, none specifically focus on leveraging user stories to generate unit tests across multiple layers, including the required mock configurations.

In contrast to existing code-dependent approaches, the Req2Test Approach generates unit tests directly from DSL user story specifications, including the necessary mock configurations. This enables test generation across multiple architecture layers without relying on an implemented system.

8 Conclusion

In this paper, we presented Req2Test, an approach leveraging LLMs to automatically generate unit test cases from user stories without requiring existing source code. Our implementation targets RESTful APIs using JUnit 5, generating unit tests for entities, use cases, and controllers with proper mocking. The approach employs a structured test planning driven by a DSL, which formalizes input specifications and enables full automation via the Req2Test tool. This DSL was specifically designed to standardize user stories and acceptance criteria. We evaluated Req2Test with 34 participants, showing strong functional effectiveness, structural quality, and internal quality. Generated tests covered normal flows, boundaries, and exceptions, adhered to architectural standards, and achieved 91.75% average code coverage with high mutation scores. Using EvoSuite as baseline, Req2Test consistently achieved higher mutation scores, especially in the UseCase package, while matching or exceeding coverage and supporting layers EvoSuite could not. In scalability tests with QuixBugs, Req2Test reached near-optimal coverage and autonomously detected 72.8% of injected faults, confirming its high-quality, reproducible test suites and practical applicability. Future works, we plan to extend Req2Test to support automated generation of integration tests. We also aim to explore direct natural language translation into the DSL, enabling a more seamless and end-to-end automated testing pipeline.

¹¹Zenodo Supplementary Materials. GitHub: <https://github.com/specgentest/req2test-tool>, <https://github.com/specgentest/dataset-api>

Acknowledgements

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001.

References

- [1] Parsa Alian, Noor Nashid, Mobina Shahbandeh, Taha Shabani, and Ali Mesbah. 2025. Feature-driven end-to-end test generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 450–462. doi:10.1109/ICSE55347.2025.00141.
- [2] M. Moein Almasi, Hadi Hemmati, Gordon Fraser, Andrea Arcuri, and Janis Benefelds. 2017. An industrial evaluation of unit test generation: finding real faults in a financial application. In *2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*, 263–272. doi:10.1109/ICSE-SEIP.2017.27.
- [3] Nahid Anwar and Susmita Kar. 2019. Review paper on various software testing techniques & strategies. *Global Journal of Computer Science and Technology*, 19, 2, 43–49.
- [4] Victor R. Basili, Gianluigi Caldiera, and Dieter H. Rombach. 1994. The goal question metric approach. In John J. Marciniak, (Ed.) *John Wiley & Sons*, 528–532.
- [5] Julian M Bass, Sarah Beecham, Mohammad Abdur Razzak, Clodagh Nie Canna, and John Noll. 2018. An empirical study of the product owner role in scrum. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings*, 123–124.
- [6] Luis A. Cisneros, Catarina I. Reis, Marisa Maximiano, and José A. Quiña. 2018. An experimental evaluation of itl, tdd and bdd.
- [7] Gabriele De Vito, Gabriele Vassallo, Fabio Palomba, and Filomena Ferrucci. 2024. Agora: an approach for generating acceptance test cases from use cases. In *2024 50th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, 126–133. doi:10.1109/SEAA64295.2024.00027.
- [8] Jannik Fischbach, Tobias Springer, Julian Frattini, Henning Femmer, Andreas Vogelsang, and Daniel Mendez. 2021. Fine-grained causality extraction from natural language requirements using recursive neural tensor networks. In *2021 IEEE 29th International Requirements Engineering Conference Workshops (REW)*. IEEE, 60–69.
- [9] Jannik Fischbach, Andreas Vogelsang, Dominik Spies, Andreas Wehrle, Maximilian Junker, and Dietmar Freudenstein. 2020. Specmate: automated creation of test cases from acceptance criteria. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, 321–331. doi:10.1109/ICST46399.2020.00040.
- [10] Jannik Fischbach et al. 2023. Automatic creation of acceptance tests by extracting conditionals from requirements: nlp approach and case study. *Journal of Systems and Software*, 197, 111549.
- [11] Jannik Fischbach et al. 2023. Automatic creation of acceptance tests by extracting conditionals from requirements: nlp approach and case study. *Journal of Systems and Software*, 197, 111549. doi:https://doi.org/10.1016/j.jss.2022.111549.
- [12] Matheus França and Fabiane Benitti. Req2test: an approach for the automatic generation of unit tests from user stories using llms. Zenodo, (Dec. 2025). doi:10.5281/zenodo.17917984.
- [13] Gordon Fraser and Andrea Arcuri. 2011. Evosuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE '11)*. Association for Computing Machinery, Szeged, Hungary, 416–419. ISBN: 9781450304436. doi:10.1145/2025113.2025179.
- [14] Norbert E Fuchs and Rolf Schwitter. 1995. Specifying logic programs in controlled natural language. *arXiv preprint cmp-lg/9507009*.
- [15] Theodore D. Hellmann, Abhishek Sharma, Jennifer Ferreira, and Frank Maurer. 2012. Agile testing: past, present, and future – charting a systematic map of testing in agile software development. In *2012 Agile Conference*, 55–63. doi:10.1109/Agile.2012.8.
- [16] Xinyi Hou et al. 2024. Large language models for software engineering: a systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33, 8, 1–79.
- [17] ISO/IEC/26515. 2018. Iso/iec/ieee international standard - systems and software engineering – developing information for users in an agile environment. *ISO/IEC/IEEE 26515:2018(E)*, 1–32. doi:10.1109/IEEESTD.2018.8584455.
- [18] Marko Ivanković, Goran Petrović, René Just, and Gordon Fraser. 2019. Code coverage at google. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2019)*. Association for Computing Machinery, Tallinn, Estonia, 955–963. ISBN: 9781450355728. doi:10.1145/3338906.3340459.
- [19] Kevin Jalbert and Jeremy S. Bradbury. 2012. Predicting mutation score using source code and test suite metrics. In *2012 First International Workshop on Realizing AI Synergies in Software Engineering (RAISE)*, 42–46. doi:10.1109/RAISE.2012.6227969.
- [20] D. Janzen and H. Saiedian. 2005. Test-driven development concepts, taxonomy, and future direction. *Computer*, 38, 9, 43–50. doi:10.1109/MC.2005.314.
- [21] Myeongsoo Kim, Tyler Stennett, Saurabh Sinha, and Alessandro Orso. 2025. A multi-agent approach for rest api testing with semantic graphs and llm-driven inputs. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 1409–1421. doi:10.1109/ICSE55347.2025.00179.
- [22] Priyanka Kulkarni and Yashada Joglekar. 2014. Generating and analyzing test cases from software requirements using nlp and hadoop. *International Journal of Current Engineering and Technology*, 4, 6, 3934–3937.
- [23] Huaizhong Li and C Peng Lam. 2007. Software test data generation using ant colony optimization. *International Journal of Computer and Information Engineering*, 1, 1, 137–140.
- [24] Yihao Li, Pan Liu, Haiyang Wang, Jie Chu, and W Eric Wong. 2025. Evaluating large language models for software testing. *Computer Standards & Interfaces*, 93, 103942.
- [25] Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. 2017. Quixbugs: a multi-lingual program repair benchmark set based on the quixey challenge. In *Proceedings Companion of the 2017 ACM SIGPLAN international conference on systems, programming, languages, and applications: software for humanity*, 55–56.
- [26] Ggaliwango Marvin, Nakayiza Hellen, Daudi Jjingo, and Joyce Nakatumba-Nabende. 2024. Prompt engineering in large language models. In *Data Intelligence and Cognitive Informatics*. I. Jeena Jacob, Selwyn Piramuthu, and Przemyslaw Falkowski-Gilski, (Eds.) Springer Nature Singapore, Singapore, 387–402. ISBN: 978-981-99-7962-2.
- [27] Glenford J Myers, Tom Badgett, Todd M Thomas, and Corey Sandler. 2004. *The art of software testing*. Vol. 2. Wiley Online Library.
- [28] Carlos Pacheco and Michael D Ernst. 2007. Randoop: feedback-directed random testing for java. In *Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion*, 815–816.
- [29] Prerana Pradeepkumar Rane. 2017. *Automatic generation of test cases for agile using natural language processing*. Ph.D. Dissertation. Virginia Tech.
- [30] Arash Sabbaghi and Mohammad Reza Keyvanpour. 2017. State-based models in model-based testing: a systematic review. In *2017 IEEE 4th international conference on knowledge-based engineering and innovation (kbei)*. IEEE, 0942–0948.
- [31] Valdivino Alexandre de Santiago Júnior and Nandamudi Lankalapalli Vijaykumar. 2012. Generating model-based test cases from natural language requirements for space application software. *Software Quality Journal*, 20, 77–143.
- [32] Sina Shamsiri, René Just, José Miguel Rojas, Gordon Fraser, Phil McMinn, and Andrea Arcuri. 2015. Do automatically generated unit tests find real faults? an empirical study of effectiveness and challenges. In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE '15)*. IEEE Press, Lincoln, Nebraska, 201–211. ISBN: 9781509000241. doi:10.1109/ASE.2015.86.
- [33] Mary Shaw and David Garlan. 1996. *Software architecture: perspectives on an emerging discipline*. Prentice-Hall, Inc.
- [34] Ravi Prakash Verma and Md Rizwan Beg. 2013. Generation of test cases from software requirements using natural language processing. In *2013 6th International Conference on Emerging Trends in Engineering and Technology*. IEEE, 140–147.
- [35] Chunhui Wang, Fabrizio Pastore, Arda Goknil, and Lionel C. Briand. 2022. Automatic generation of acceptance test cases from use case specifications: an nlp-based approach. *IEEE Transactions on Software Engineering*, 48, 2, 585–616. doi:10.1109/TSE.2020.2998503.
- [36] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: survey, landscape, and vision. *IEEE Transactions on Software Engineering*, 50, 4, 911–936. doi:10.1109/TSE.2024.3368208.
- [37] Wen-Li Wang, Ye Wu, and Mei-Hwa Chen. 1999. An architecture-based software reliability model. In *Proceedings 1999 Pacific Rim International Symposium on Dependable Computing*. IEEE, 143–150.
- [38] Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, Anders Wesslén, et al. 2012. *Experimentation in software engineering*. Vol. 236. Springer.
- [39] Matt Wynne and Aslak Hellesoy. 2012. *The Cucumber Book: Behaviour-Driven Development for Testers and Developers*. Pragmatic Bookshelf. ISBN: 1934356808.
- [40] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and improving chatgpt for unit test generation. *Proceedings of the ACM on Software Engineering*, 1, FSE, 1703–1726.
- [41] Jian Zhang, Xu Wang, Hongyu Zhang, Hailong Sun, Kaixuan Wang, and Xudong Liu. 2019. A novel neural source code representation based on abstract syntax tree. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 783–794.